

- Um ein sauberes, sicheres und modulares Python-Projekt zu erstellen, das PDFs mit A-Trust ASignSealQualified-Zertifikaten signiert, können Sie die folgenden Schritte befolgen:
- “ein sauberes, sicheres und modulares Python-Projekt, das PDFs mit A-Trust ASignSealQualified-Zertifikaten signiert”

1 Voraussetzungen

- **Python 3.7+**
- **pycryptodome**: Für kryptografische Operationen.
- **PyPDF2** oder **pikepdf**: Zum Bearbeiten von PDF-Dokumenten.
- **requests**: Für HTTP-Anfragen.
- **pyhsm** oder **pkcs11**: Für die Interaktion mit Hardware-Sicherheitsmodulen (HSM).([GitHub](#), [apryse-docs](#), [DocuSeal](#))

2 ✂ Architektur des Projekts

- Die Projektstruktur könnte wie folgt aussehen:

```
pdf_signer/
├── certs/
│   ├── private_key.pem
│   └── certificate.pem
├── src/
│   ├── __init__.py
│   ├── signer.py
│   └── utils.py
├── tests/
│   ├── test_signer.py
│   └── test_utils.py
├── requirements.txt
└── README.md
```

3 Beispielcode: PDF signieren

- Hier ein einfaches Beispiel, wie Sie ein PDF-Dokument mit einem Zertifikat signieren können:

```
from PyPDF2 import PdfReader, PdfWriter
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA
import requests
```

```
def sign_pdf(input_pdf_path, output_pdf_path, cert_path, key_path):
    # PDF einlesen
    reader = PdfReader(input_pdf_path)
```

```

writer = PdfWriter()

# Zertifikat und privaten Schlüssel laden
with open(cert_path, 'rb') as f:
    cert = f.read()
with open(key_path, 'rb') as f:
    key = f.read()

# SHA-256 Hash des Dokuments berechnen
doc_hash = SHA256.new(reader.pages[0].extract_text().encode('utf-8'))

# Signatur erstellen
private_key = RSA.import_key(key)
signer = pkcs1_15.new(private_key)
signature = signer.sign(doc_hash)

# Signatur zum PDF hinzufügen (z.B. als Anmerkung)
# Hier müsste Code hinzugefügt werden, um die Signatur im PDF zu platzieren

# Geändertes PDF speichern
with open(output_pdf_path, 'wb') as f:
    writer.write(f)

return signature

```

- Dieser Code ist ein Ausgangspunkt und muss angepasst werden, um die Signatur korrekt im PDF-Dokument zu platzieren.

4 Teststrategie

- Für die Tests können Sie pytest verwenden. Ein einfacher Test könnte wie folgt aussehen:

```

import pytest
from src.signer import sign_pdf

def test_sign_pdf():
    input_pdf = 'test.pdf'
    output_pdf = 'signed_test.pdf'
    cert = 'certs/certificate.pem'
    key = 'certs/private_key.pem'

    signature = sign_pdf(input_pdf, output_pdf, cert, key)

    assert signature is not None

```

Weitere Assertions, um die Korrektheit der Signatur zu überprüfen

5 Weiterführende Ressourcen

- [A-Trust ASignSealQualified GitHub Repository](#)
- [PyPDF2 Dokumentation](#)
- [pycryptodome Dokumentation](#)

6 Fazit

- Dieses Setup bietet eine modulare und sichere Grundlage für das Signieren von PDFs mit A-Trust ASignSealQualified-Zertifikaten. Durch die Verwendung von bewährten Bibliotheken und einer klar strukturierten Architektur wird die Wartbarkeit und Erweiterbarkeit des Projekts gewährleistet. ([GitHub](#), [apryse-docs](#))